IMPLEMENTATION REPORT

Kelly Terminal

A Kelly Criterion position-sizing calculator and Monte Carlo simulator across five live-monitored instruments, delivered as a static single-page application.

Field	Value
Repository	github.com/xyb3rpunq/Kelly-Criterion
Live deployment	xyb3rpunq.github.io/Kelly-Criterion/
Report date	7 August 2026
Stack	React 18 · Vite 8 · TailwindCSS 3.4 · Framer Motion · Recharts 3
Source size	4,266 lines across 24 JavaScript/JSX modules
Tests	28 unit tests over the pure maths layer — all passing
Commits	5 feature commits on main, each built and verified before push
Languages	English and Bahasa Indonesia, full parity including generated prose
Licence	MIT

Scope note

The brief specified a gold-only tool. During the build the scope was extended on request to five monitored instruments (DXY, XAU/USD, XAG/USD, USOIL, BTC/USD) with real market data, a bilingual interface, and automated deployment. Everything specified in the original brief was delivered; this report covers both the original scope and the extensions.

01 Executive summary

The delivered application takes a trade's geometry — entry, stop and target — together with the user's own estimate of how often that setup wins, derives the reward-to-risk odds, applies the Kelly Criterion, simulates 24 equity paths over 120 trades, and generates a written risk assessment in the register of an internal desk memo.

Three things distinguish it from a conventional position-size calculator. First, the conclusion section is **composed from live simulation output** rather than being static prose with numbers substituted in — including a sensitivity grid that re-runs the entire simulation at plus and minus five percentage points on the assumed win rate, on the same random seed, so the comparison isolates the assumption rather than the draw. Second, all market data is **genuinely live** where a browser-reachable feed exists, and explicitly labelled as cached where one does not. Third, the interface deliberately avoids the decorative dishonesty common to web3-styled tools: there is no wallet button, because the page connects to no chain.

Deliverable	Status	Evidence
Single-page React + Vite application	Delivered	Builds clean, deployed and reachable
Custom Tailwind theme (no default palette)	Delivered	tailwind.config.js replaces colors wholesale
Framer Motion load sequence + micro-interactions	Delivered	Staggered hero and panel reveal

Deliverable	Status	Evidence
Recharts Monte Carlo visualisation	Delivered	24 paths, median, ruin line, auto log axis
Signature element (Kelly gauge)	Delivered	270-degree radial arc, 4 zones, live comet
Auto-generated institutional risk memo	Delivered	5 subsections, all values computed live
Pure, testable maths module	Delivered	src/lib/kelly.js, 28 tests
Five live-monitored instruments	Delivered	4 live feeds + 1 documented server-side
Bilingual EN / ID	Delivered	Verified zero English leakage in ID mode
GitHub Pages + Vercel deployment	Delivered	Actions workflow green; vercel.json present
Honest UX labelling	Delivered	Per-instrument live/cached badges, no fake wallet
Accessibility baseline	Delivered	27/27 controls named, focus visible, reduced-motion

02 The mathematics

All computation lives in `src/lib/kelly.js` as pure functions — no React, no DOM, no hidden state, deterministic given their arguments including the Monte Carlo, which takes an explicit seed. This is what makes the layer testable in isolation.

```
edge = p·b − q                expectancy in R-multiples,  q = 1 − p
f*   = edge / b               optimal fraction of capital
p_be = 1 / (1 + b)            win rate required to break even
```

Design decisions worth recording

Decision	Rationale
Negative f* floors at zero	The formula's advice to bet the opposite side is meaningless for a directional trade the user has already chosen. Returning a negative size would be actively misleading.
f* capped just below 1	Prevents a full-Kelly path multiplying by exactly zero and producing a hard, unrecoverable wipeout that the compounding model cannot represent.
Multiplicative sizing in the simulation	Each trade risks a fixed fraction of current equity, which is what Kelly actually assumes. This is why a losing streak shrinks the next bet rather than marching linearly to zero.
Ruined paths freeze	A path reaching 50% of starting capital stops trading. A desk that halves its NAV is shut down, not left to recover — allowing recovery would flatter the ruin statistics.
Median taken per time-step	The median line is a genuine cross-sectional median across paths at each step, not the middle path selected from the bundle, which would be one lucky sequence.
Stop on the wrong side is rejected	Treated as user error and surfaced with a field-level message, rather than silently producing a negative b that would corrupt everything downstream.
2% house cap layered on top	Allocation committees overlay a hard per-trade limit on model output. The binding constraint is always the lower of the two, and the memo says which one is binding.

Risk tier classification

Condition	Tier	Reasoning
b invalid	Incomplete Setup	Geometry does not resolve
edge ≤ 0	No Edge — Do Not Size	Kelly returns no positive allocation
f* < 5%	Marginal Edge	Indistinguishable from estimation error
5% ≤ f* ≤ 25%	Edge Present	Supports conventional fractional sizing
f* > 25%	Edge Overstated	Signals an optimistic p, not a rare opportunity

The top tier is the pedagogically important one

The default setup — a 1:2 reward-to-risk trade with an assumed 55% win rate — produces a full Kelly of 32.5% of NAV and immediately trips the Edge Overstated classification. That is not a defect of the defaults. A model demanding a third of the account on one trade is almost always reporting an optimistic assumption, and the tool is built to say so on first load rather than flatter the user.

03 Market data architecture

Every candidate source was probed for CORS support before selection, because a static page served from GitHub Pages can only reach APIs that permit cross-origin browser requests. The results determined the architecture.

Instrument	Source	Live	Notes
DXY	Computed from api.fxratesapi.com	Yes	Official ICE geometric-weight formula over six currencies
XAU/USD	api.gold-api.com	Yes	Spot gold, sub-minute updates
XAG/USD	api.gold-api.com	Yes	Spot silver
USOIL	Yahoo CL=F, server-side	No	No free CORS-enabled crude feed exists — see below
BTC/USD	Binance BTCUSDT	Yes	Falls back to Coinbase where geo-blocked

DXY is computed, not proxied

Rather than substituting a correlated instrument, the dollar index is calculated in the browser from live currency rates using the index's actual published definition:

```
DXY = 50.14348112
    × EURUSD^-0.576 × USDJPY^0.136 × GBPUSD^-0.119
    × USDCAD^0.091  × USDSEK^0.042 × USDCHF^0.036
```

Validated against the real index

During development the computed value was checked against the published DX-Y.NYB print: **99.956 computed against 99.962 published**, a difference of six thousandths of an index point. The formula is the definition, so this is a correctness check rather than a correlation.

Why crude oil is handled differently

Every free CORS-enabled WTI source was tested and rejected. Recording the failures here because the negative result is the justification for the added complexity:

Candidate	Outcome
Yahoo Finance	Serves data correctly but sends no Access-Control-Allow-Origin header
Stooq	Now gated behind JavaScript-based bot detection
allorigins proxy	Returned HTTP 500
corsproxy.io	Server-side requests paywalled
gold-api.com	No oil symbols (probed WTI, OIL, BRENT, USOIL, CL)
fxratesapi	Currencies and precious metals only

Crude is therefore fetched server-side by one of two paths, chosen automatically at runtime. On Vercel a serverless function at `/api/quotes` proxies Yahoo and the value is genuinely live. On GitHub Pages the function does not exist, the client detects the 404 and falls back to `market-cache.json`, baked into the deployed artifact and refreshed every thirty minutes by the scheduled workflow. The instrument card is labelled **cached** in that state rather than being presented as a tick feed.

A subtle correctness issue: the futures basis

Yahoo's gold and silver symbols are futures (GC=F, SI=F) while the browser reads spot prices. Carrying the futures previous-close straight over would have baked the basis — roughly 1.3% on gold at the time of building — into every day-change percentage shown. The build script rescales the previous close into spot-equivalent terms before writing it, so the percentage reflects the market rather than the contract difference.

The scheduled refresh writes into the deployed artifact rather than committing back to the repository, so keeping crude fresh never pollutes the commit history.

04 The conclusion section

This was the part of the brief singled out as most important, and it received the most attention. The section renders as an internal risk memorandum. **No sentence describing results is static** — every figure and every clause is composed from the live simulation, in whichever language is active.

#	Subsection	Content
01	Executive summary	Two to three generated sentences: assumed win rate against the odds, expectancy in R-multiples, distance above or below break-even, the implied and applied fractions, and the simulation outcome. Branches into three distinct compositions for invalid geometry, non-positive edge, and positive edge.
02	Risk-adjusted sizing	Full, half and quarter Kelly as percentages and dollar amounts, each marked against the 2% house cap with the multiple by which it exceeds the limit, plus an explanation of why desks almost never run full Kelly.
03	Scenario sensitivity	Three rows at p–5pp, p and p+5pp. Each row re-runs the complete Monte Carlo at that probability on the same seed, so the median-final column isolates the effect of the assumption rather than a different random draw. The prose quantifies the resulting swing in f* and states plainly when the downside case removes the edge entirely.
04	Key risk factors	Model risk, path dependency (quoting the actual average maximum drawdown and worst observed path), regime and correlation risk naming the selected instrument, and execution risk. A fifth item appears automatically when the house cap is binding.
05	Verdict	A single bold line in desk format quoting the capped size, followed by the tier rationale.

Worked example of generated output

Produced by the running application at the default setup — XAU/USD, 1:2 odds, assumed 55% win rate, half Kelly, $10,000 of capital:

> At an assumed win rate of 55.0% against reward-to-risk odds of 1:2.00, XAU/USD carries a positive expectancy of +0.65R per trade — 21.7 percentage points above the 33.3% break-even win rate these odds demand. Full Kelly implies 32.50% of NAV per trade; the selected half Kelly allocation is 16.25%, or $1,625 of risk on $10,000 of capital. Across 24 simulated paths of 120 trades, median terminal equity is $20,217,283, with 75% of paths finishing profitable, 25% reaching the –50% ruin threshold, and an average maximum drawdown of 64%.
>
> **VERDICT: EDGE OVERSTATED — RE-EXAMINE INPUTS — position size capped at 2.00% NAV ($200) pending live confirmation of edge.**

Note what the memo does with those numbers. A 25% probability of ruin and a 64% average drawdown at half Kelly are the direct consequence of an optimistic win-rate assumption, and the verdict cuts the position to the 2% house limit rather than endorsing the model's 16.25%. The tool argues with its own output.

05 Design and interface

The brief explicitly ruled out three over-used directions: cream-and-terracotta with serif type, generic neon-on-black, and broadsheet hairline rules. The identity was instead derived from the subject matter — gold as a metal, the on-chain layer as electric violet.

Token	Value	Role
void	#05060A	Page base — near-black, shifted blue rather than neutral
panel	#0B0E14	Panel surface
gold	#D4AF37 → #F4E4A6	Accent 1 — the metal
chain	#6E5BFF	Accent 2 — the on-chain layer
mint	#00E5C7	Positive signal
danger	#FF4D6D	Ruin, negative edge

Choice	Implementation
Palette replaced, not extended	The Tailwind theme overrides the colors key wholesale, so no default Tailwind colour is reachable from any component — the design cannot drift back to slate-500 by accident.
Gradient panel edges	Panels use a 1px gradient border painted with a mask-composite trick rather than a solid border, so an edge reads as metal catching light.
Restrained radii	Nothing rounds past 6px. The result should register as an instrument, not a toy.
Typography with intent	Space Grotesk for display, JetBrains Mono for every figure with tabular-nums so changing digits never shift the layout, Inter for body copy.
Motion where it earns its place	One orchestrated load sequence plus hover micro-interactions. Not animation everywhere, which is the giveaway of generated design. All of it disappears under prefers-reduced-motion.

Signature element — the Kelly gauge

A 270-degree radial arc in which everything drawn is real data rather than decoration. The four risk-zone bands sit on the same axis as the indicator, so their boundaries are directly comparable. Ghost notches mark where the two unselected fraction presets would land. A halo behind the dial brightens with the applied fraction, so the whole instrument responds rather than just the pointer.

A comet runs the live arc using a CSS dash animation on a path carrying `pathLength="100"`. Normalising the path length means one animation cycle is exactly one traversal whether f* is 1% or 40% — without it, a short arc would barely show the dash. Because it is pure CSS it costs nothing per frame in React.

> **Honesty rules applied to a web3 theme**
>
> A web3 aesthetic invites elements that lie for decoration. None were used. There is no Connect Wallet button, because the page connects to no chain and holds no keys — the status pill instead reports what it does connect to, public price APIs, and their real state. Every instrument card carries its own live or cached badge, source name, and observation age in seconds. Sparklines are labelled as session history rather than the trading day. When the equity chart switches to a logarithmic axis it prints that fact beneath itself. And the probability panel states in the interface that p is the user's assumption, that nothing in the app measures or forecasts it, and that every downstream figure inherits its error.

06 Verification and defects found

The application was audited in a live browser rather than assumed correct from a clean build. Three genuine defects surfaced and were fixed; each now has regression coverage.

#	Defect	Diagnosis and fix
01	Equity chart unreadable	Compounding half Kelly on a positive edge over 120 trades takes $10,000 past $10,000,000. On a linear axis every path but the luckiest collapsed onto the zero line and the y-axis printed values like $8414.81M. The axis now switches to logarithmic once the spread passes two decades — standard practice for equity curves — and prints which scale is in use so the reader is never switched silently. The compact currency formatter was extended through billions and trillions.
02	Sizing figure could freeze on a stale value	The animated number component drove its tween with requestAnimationFrame, which does not fire in a backgrounded tab. Reproduced with the gauge reading 0.00% while the ladder beside it correctly showed 49.25%. For a position-sizing readout that is the worst available failure mode. The component now skips the tween when the document is hidden and carries a timeout that forces convergence if frames stop mid-animation.
03	Direction toggle invalidated the setup	Flipping buy to sell left the stop and target on the wrong sides of entry. Direction changes now mirror both levels around entry, preserving the exact risk and reward distances already chosen. Verified to round-trip: buy to sell and back restores the original figures precisely, with b holding at 2.00 throughout.

Checks performed and passed

Area	Result
Unit tests	28 passing over the pure maths layer
Production build	Clean, no errors or unresolved imports
Dependency audit	0 vulnerabilities (Vite, Vitest and Recharts upgraded to clear 5)
Console at runtime	No errors in a clean mount
Numeric integrity	No NaN, Infinity or undefined reached the DOM in any state exercised
Risk tiers	All five render correctly, including both boundary extremes of p
Instrument switching	All five seed a volatility-scaled setup at their own precision
Invalid input	Wrong-side stop, empty entry and zero capital all degrade with a message
Mobile at 375px	No horizontal page overflow; both memo tables scroll in their containers
Accessibility	27 of 27 interactive elements carry an accessible name
Bilingual parity	Zero English leakage found scanning the Indonesian render
Live deployment	HTTP 200, correct base paths, all assets and the data cache resolve

Unit test coverage

The 28 tests target the places where a silent error would be most expensive: the textbook Kelly case must return exactly 20% for p=0.6 and b=1; the break-even boundary must return exactly zero; a negative edge must floor rather than imply a reverse bet; the seeded generator must be reproducible, differ across seeds and stay within its range; a ruined path must never recover; worst must not exceed median which must not exceed best; full Kelly must produce heavier drawdowns than quarter Kelly; and the house cap must bind when the model asks for more. Five further tests were added from the browser audit, covering blank fields, the mirroring identity, extremes of p, and the capital floor.

07 Delivery and deployment

Target	Configuration	Data quality
GitHub Pages (primary)	Actions workflow on push to main plus a 30-minute schedule. Runs tests, refreshes the market snapshot, builds with the base path set to /Kelly-Criterion/, deploys.	Four instruments live; crude up to 30 minutes old and labelled cached
Vercel (optional)	vercel.json committed. Deploying activates the /api/quotes serverless function, detected automatically by the client with no configuration.	All five instruments live

No API keys exist anywhere in the project. Every source is a free public endpoint, which is what makes a static deployment viable in the first place.

Repository contents

Path	Purpose
src/lib/kelly.js	Pure maths — Kelly, Monte Carlo, sensitivity, tiers, sizing ladder
src/lib/kelly.test.js	28 unit tests
src/lib/market.js	Data layer, source selection, DXY computation
src/lib/i18n.jsx	English and Indonesian dictionaries, memo prose generators
src/components/	Twelve components plus three UI primitives
src/hooks/	Market polling, debounce, language context
scripts/fetch-market.mjs	Build-time market snapshot with basis correction
api/quotes.js	Vercel serverless proxy
.github/workflows/deploy.yml	Test, refresh, build, deploy
README.md / README.id.md	Bilingual documentation, twelve sections each

Known limitations

Limitation	Detail
Crude is not live on GitHub Pages	A consequence of there being no free CORS-enabled WTI feed, not an implementation shortcut. Deploying to Vercel resolves it. The interface labels the state honestly either way.
Bundle size ~205 KB gzipped	Dominated by Recharts. Acceptable for a single-page tool; lazy-loading the chart would be the first optimisation if it mattered.
The model assumes i.i.d. trades	Real trades clustered around the same macro catalyst are correlated. The memo states this explicitly as a risk factor rather than leaving it implicit.
Free public price APIs	May be delayed, rate-limited or briefly wrong. Each instrument fails independently and reports itself unavailable rather than taking the page down.

Disclaimer

The delivered application is an educational simulator. It is not financial, investment or trading advice, and it is not a research product. The win probability is a subjective figure entered by the user; it is not measured, forecast or validated, and every conclusion the application produces inherits its error. Simulated results are not indicative of future performance. The project is an independent open-source exercise with no affiliation to, endorsement by, or connection with any bank, fund, exchange, brokerage or financial institution — including Citadel or any other firm whose house style the tone of the generated memo may resemble. Market prices come from free public APIs and must not be used for execution.